ATELIER 11 La tournée de Noël

Algorithmie comparative - J5 - Vendredi 13 novembre 2026, 12h25 puis 13h40 - 70 minutes, en binôme : 15 avant le déjeuner, puis 13h40-14h35

La question à laquelle l'atelier répond

Les tournées calculées font 40 % de kilomètres de plus que celles que les livreurs tracent à la main (ticket T-0531), et le pic de Noël loue dix utilitaires (pièce 06) ; le scénario de l'atelier fait passer les vélos cargo à quarante arrêts par vague. Dans quel ordre visiter quarante arrêts : plus proche voisin, 2-opt ou recuit simulé ?

Ce que vous produisez : adr/GRILLE-ADR-008.md et adr/ADR-008-tournees.md (carnet), deux CSV dans bench/resultats (*-tournees.csv, *-noel.csv).

Le rendu attendu

En fin d'atelier : le code, les mesures et la grille passent le correcteur, et l'ADR existe en premier jet (chaque rubrique en une à trois lignes, avec vos chiffres et le nom de vos CSV). Le correcteur sans KO avant 17h00, le commit du binôme à 17h00. Ensuite, l'ADR s'améliore chez vous jusqu'au dépôt du carnet, le 1er décembre.

Avant 17h00, le correcteur de l'atelier répond sans KO (extrait, capturé quand c'était l'atelier 12 ; chez vous, ADR-008) :

```
OK  À COMPLÉTER : le 2-opt ne rallonge jamais la tournée du plus proche voisin
OK  Sur neuf arrêts, le 2-opt est à moins de 3 % de l'optimum
OK  Le recuit fait au moins aussi bien que le plus proche voisin, et pareil deux fois de suite
OK  La qualité est mesurée
OK  La grille comparative de l'ADR-009 est remplie
9 contrôles, 0 KO
```

Avant de commencer. Les distances entre arrêts se calculent par la route, avec votre Dijkstra de l'atelier 10 (bench/variantes/chemins.ts ; s'il est faux ou inachevé : annexe), une fois, et se gardent dans data/jeux/moyenne/cache-tournees-40.json : la première commande attend de quelques secondes à quelques dizaines de secondes selon le poste. Si le réseau routier manque : annexe. Créez l'ADR-008 du binôme, que vous remplissez dès le sous-objectif 1 :

Windows	`Copy-Item adr\ADR-000-gabarit.md adr\ADR-008-tournees.md`
macOS et Linux	`cp adr/ADR-000-gabarit.md adr/ADR-008-tournees.md`

Pour chaque sous-objectif, vous trouvez la première commande et ce que vous devez voir ; les suivantes sont dans le README du dépôt (tableau des commandes de la journée 5) et dans les fiches atelier numéro 8 à 10. Les barèmes de la grille sont en annexe.

Les sous-objectifs

1. Reproduire le ticket consigne 1 de la slide

Pourquoi. Voir ce que l'ordre d'affectation coûte en kilomètres sur des tournées de quarante arrêts, puis ce que le plus proche voisin en fait déjà.

Première action. Lancez la qualité sur cinq tournées, puis relevez en rubrique 1 de l'ADR-008 les kilomètres de la ligne legacy (l'ordre d'affectation) et de la ligne voisin.

Les trois systèmes	`npm run noel -- --tournees 5`

Vous devez voir. La préparation des distances par la route (41 recherches par tournée, une fois), puis une ligne par méthode. La ligne deuxopt rend les kilomètres du voisin tant que le gain n'est pas écrit. Les kilomètres sont les mêmes sur tous les postes.

```
[tournees] distances prêtes : 60 tournées en 8,4 s, gardées dans cache-tournees-40.json
variante    km totaux   écart au meilleur   temps total
legacy        1 152,7   +211,6 %                     -
voisin          432,1   +16,8 %                  0 ms
deuxopt         432,1   +16,8 %                  0 ms   (0 échanges)
recuit          370,0   meilleur                14 ms
```

2. Compléter le gain du 2-opt, lancer les tests consigne 2 de la slide

Pourquoi. Le gain d'un échange, en quatre distances : c'est le seul À COMPLÉTER ; l'inversion du morceau, s'il est positif, est donnée.

Première action. Dans bench/variantes/tournees.ts, ligne 91, écrivez le gain d'un échange : la longueur des deux arêtes retirées, (a, b) et (c, e), moins celle des deux ajoutées, (a, c) et (b, e). Lancez `npm test`, puis `npm run verifier -- 11`.

Windows	`code bench\variantes\tournees.ts`
macOS et Linux	`code bench/variantes/tournees.ts`

Vous devez voir. Les tests des tournées passent ; le premier contrôle du correcteur le redit sur dix tournées.

```
✔ le 2-opt trouve l'ordre optimal du rectangle depuis un ordre croisé (0.7222ms)
✔ le 2-opt ne rallonge jamais le plus proche voisin, sur 20 tournées au sort (3.3398ms)
```

3. Mesurer les trois méthodes consigne 3 de la slide

Pourquoi. Le temps, pour 1, 5 et 20 tournées : le plus proche voisin et le 2-opt en millisecondes, le recuit avec son budget de 60 000 pas.

Première action. Mesurez les trois méthodes avec le harness ; relevez les trois médianes à 20 tournées.

Les trois systèmes

```
npm run bench -- --sujet tournees --variantes voisin,deuxopt,recuit --tailles 1,5,20 --repetitions 3 --echauffement 1
```

Vous devez voir. Le voisin et le 2-opt mettent moins d'une milliseconde par tournée, le recuit bien plus. La somme de contrôle compte les tournées.

```
variante        taille   duree_ms   memoire_ko
voisin              20       0.14           18
deuxopt             20       0.68           71
recuit              20      34.44         3675
```

4. Mesurer la qualité consigne 4 de la slide

Pourquoi. Le critère du ticket : les kilomètres. Sur quarante arrêts, personne ne connaît l'optimum ; sur neuf, la force brute le donne.

Première action. Lancez la qualité sur les 60 tournées, puis sur neuf arrêts contre l'optimum ; relevez les kilomètres de chaque méthode, ce que la flotte économise, l'écart moyen à l'optimum.

Les trois systèmes

```
npm run noel
npm run noel -- --arrets 9
```

Vous devez voir. Les kilomètres par méthode, l'écart au meilleur des quatre, la ligne de la flotte ; puis l'écart à l'optimum sur neuf arrêts (ici, deux des quatre lignes).

```
variante    km totaux   écart au meilleur   temps total
legacy       16 267,5   +233,8 %                     -
voisin        5 706,7   +17,1 %                  1 ms
deuxopt       4 966,6   +1,9 %                   4 ms   (1 114 échanges)
recuit        4 873,8   meilleur               111 ms
[noel] sur la flotte, 60 tournées : 11 301 km de moins par jour que l'ordre d'affectation avec le 2-opt
(11 394 km avec le recuit) ; 271,1 km par tournée dans l'ordre d'affectation, 82,8 km avec le 2-opt
```

```
  voisin   : +10,2 % en moyenne, optimum trouvé 3 fois sur 60 (au pire +29,0 %)
  deuxopt  : +0,4 % en moyenne, optimum trouvé 47 fois sur 60 (au pire +5,5 %)
```

5. Remplir la grille, écrire l'ADR-008, vérifier consignes 5, 6 de la slide

Pourquoi. Les poids du ticket sont imposés : temps 3, mémoire 1, lisibilité 2, robustesse 3, coût 1. Les barèmes, écrits avant de mesurer, sont en annexe et sur la slide de relance ; la robustesse se lit sur l'écart moyen à l'optimum (neuf arrêts).

Première action. Copiez la grille, notez les trois méthodes, puis écrivez l'ADR-008 : sa rubrique 5 dit le seuil (à partir de combien d'arrêts, ou de kilomètres gagnés, le recuit vaut son temps) et ce qu'on n'a pas mesuré (les créneaux horaires, les arrêts de décembre). Le legacy est l'option A de la grille et se cite en rubrique 1. Préparez la décision et la question de compréhension à deux, puis lancez `npm run verifier -- 11`.

Windows

```
Copy-Item adr\grille-comparative.md adr\GRILLE-ADR-008.md
```

macOS et Linux

```
cp adr/grille-comparative.md adr/GRILLE-ADR-008.md
```

- L'ADR-008 est un premier jet ; ce qui manque se reprend à 16h35, au créneau des correcteurs, puis chez vous.
- Rubrique 2 : la graine du recuit et son budget de pas font partie du protocole ; sans eux, la mesure n'est pas rejouable.
- Rubrique 3 : les colonnes du tableau sont 1, 5 et 20 tournées, puis les kilomètres sur 60 tournées et l'écart à l'optimum sur neuf arrêts.
- Rubrique 6 : ni unité ni nom de méthode ; le correcteur donne les mots qui coincent.

Vous devez voir. Le correcteur répond sans KO, comme en première page.

La décision de l'atelier

Quelle façon d'ordonner les arrêts retenez-vous, et au-delà de quel seuil changeriez-vous de méthode ?

La question de compréhension

À quoi sert ce que vous venez de faire, en deux phrases ? Préparez-la à deux : en fin d'atelier, l'intervenant passe la demander à quelques binômes.

À 17h00, le binôme commite : git add -A puis git commit -m "J5". Sans commit, le travail du jour n'est pas évalué.

Annexe, à consulter au besoin

Les barèmes, tels que le cours 10 les pose

Les barèmes s'écrivent avant de mesurer ; la mesure vient ensuite. Dans tous les cas, justifiez la note en une phrase sous la grille : le barème dit la note, la phrase dit ce que vous avez observé.

Temps (poids 3) : La médiane à 20 tournées : 5 sous 100 ms, 4 sous 1 s, 3 sous 10 s, 2 sous 1 min, 1 au-delà.

Robustesse (poids 3) : L'écart moyen à l'optimum sur neuf arrêts : 5 sous 1 %, 4 sous 3 %, 3 sous 10 %, 2 sous 25 %, 1 au-delà. Elle se lit dans la sortie de npm run noel -- --arrets 9.

Mémoire (poids 1) : Le pic du harness, matrice comprise : 5 sous 10 Mo, 4 sous 50 Mo, 3 au-delà. Relancez la mesure une fois : si un relevé bouge d'un facteur 2 ou plus entre les deux lancements, la mémoire ne départage pas et les options prennent la même note ; deux options à moins d'un facteur 2 l'une de l'autre ont la même note.

Lisibilité (poids 2) : La meilleure mesure de la lisibilité reste de faire lire son code à quelqu'un qui ne l'a pas écrit. Ici, c'est votre cas : lisez les trois variantes de bench/variantes/tournees.ts, que vous n'avez pas écrites, et listez pour chacune les idées qu'il faut connaître pour répondre à trois questions : que fait la fonction, que se passe-t-il quand un échange n'améliore pas, où changer la graine. On part de 5 et chaque idée coûte un point.

Coût de mise en oeuvre (poids 1) : 5 si rien ne change ; 4 pour une fonction courte sans réglage ; 3 pour une fonction longue ou un réglage à tenir ; 2 pour les deux ; 1 pour une dépendance. La température, le budget de pas et la graine du recuit sont des réglages à tenir.

Les cas particuliers

npm run noel répond que le réseau routier est absent : maj-J4.zip n'est pas décompressé à la racine du dépôt :

Windows	`Expand-Archive -Path $HOME\Downloads\maj-J4.zip -DestinationPath . -Force`
macOS et Linux	`unzip -o ~/Downloads/maj-J4.zip -d .`

npm run noel s'arrête sur "ARRÊT : dijkstraTas ne rend pas les distances de dijkstraSimple" : votre Dijkstra avec tas de l'atelier 10 est faux ou inachevé. Le correcteur 11 peut aussi dire que les mètres annoncés ne sont pas ceux de la matrice. Dans les deux cas : faites un commit, décompressez correction_atelier10.zip à la racine du dépôt (il range chemins.ts complété dans correction/J4, sans rien remplacer), puis copiez correction/J4/chemins.ts vers bench/variantes/chemins.ts, puis relancez les tests (21 sur 25 au moins) et la commande :

Windows	`Expand-Archive -Path $HOME\Downloads\correction_atelier10.zip -DestinationPath . -Force`
	`Copy-Item correction\J4\chemins.ts bench\variantes\chemins.ts`
	`npm test`

macOS et Linux	`unzip -o ~/Downloads/correction_atelier10.zip -d .`
	`cp correction/J4/chemins.ts bench/variantes/chemins.ts`
	`npm test`

Où trouver un exemple dans le code

bench/variantes/tournees.ts contient le plus proche voisin et le recuit écrits en entier : le recuit montre comment on inverse un morceau de tournée et comment on calcule ce qu'un échange ajoute. bench/voyageur.ts (la démo du voyageur) contient la force brute que `--arrets 9` réutilise.

Si ça coince

- `npm run noel` s'arrête en moins d'une seconde sur "ARRÊT : dijkstraTas ne rend pas les distances de dijkstraSimple" : votre Dijkstra de l'atelier 10 est faux ou inachevé ; le remède est dans l'annexe (correction_atelier10.zip).
- Le correcteur dit que le 2-opt rallonge la tournée : le gain est compté à l'envers (ce qu'on ajoute moins ce qu'on retire). S'il rend la longueur du voisin avec 0 échange : le gain vaut toujours 0 (la ligne provisoire), ou il manque un terme dans ce qu'on retire.
- `npm test`, `npm run noel` ou le correcteur s'arrête sur "le 2-opt ne converge pas" : des échanges sont faits sans que la tournée raccourcisse. Le gain n'ajoute pas les bonnes arêtes, (a, c) et (b, e), ou il lui manque un terme.

Toujours bloqué après ces trois pistes : levez la main, ou écrivez sur Teams.

Si vous avez fini

- Donnez plus de pas au recuit (`npm run noel -- --pas 200000`) et regardez ce que les kilomètres gagnés coûtent en temps.

Rien de cette liste n'est contrôlé par le correcteur de l'atelier : ne pas le faire ne laisse aucun KO.

Les sorties montrées sont de vraies sorties, capturées en jouant la fiche atelier en cours ; vos durées seront différentes, vos kilomètres et vos nombres d'échanges, non.